

Manual completo do usuário

Referência operacional do V&V TestLab para planejar, executar e documentar testes de software com rastreabilidade completa.

1	CRUD, filtros e persistência dos dados
2	Classes, limites e casos válidos ou inválidos
3	Execuções, métricas, defeitos e rastreabilidade
4	Backup, restauração e relatório PDF A4

Sumario

Use o sumário para ir diretamente a uma tarefa. O manual segue a ordem recomendada para o trabalho da disciplina.

1. Sobre a plataforma	4
O que a plataforma faz	4
2. Iniciar a plataforma	4
Pré-requisitos	4
Primeira execução	4
Parar a plataforma	5
3. Entender a navegação	5
4. Configurar o projeto	5
5. Definir as três funcionalidades principais	6
Editar uma funcionalidade	6
Substituir uma funcionalidade	6
Limite do escopo	6
6. Criar classes de equivalência	6
7. Gerar casos por valor-limite	7
8. Criar casos manualmente	7
Técnicas disponíveis	8
9. Pesquisar e filtrar	8
Filtros disponíveis	8
10. Registrar uma execução manual	8
11. Registrar cobertura e mutação	8
Etapa funcional	8
Etapa estrutural	9
Etapa baseada em defeitos	9
12. Integrar com o repositório Python	9
Instalar no ambiente do projeto	9
Marcar os testes	9
Iniciar a ponte	9

Sincronizar	10
Executar mutação	10
Exemplo pronto: sistema de hotel	10
13. Utilizar a matriz de rastreabilidade	11
Interpretar lacunas	11
Navegar pela matriz	11
Filtros da matriz	11
14. Registrar defeitos	11
15. Gerar o relatório final	11
16. Backup e restauração	12
Criar backup	12
Restaurar backup	12
Restaurar demonstração	13
17. Persistência e privacidade	13
18. Solução de problemas	13
Os botões não respondem	13
Meus dados desapareceram	13
Não consigo criar uma funcionalidade	13
A classe não aparece no caso	13
A ponte não conecta	13
A cobertura não foi importada	13
A mutação não inicia	13
Um teste aparece como não vinculado	13
O PDF possui informações incompletas	14
19. Fluxo recomendado para o trabalho	14
Fase 1	14
Teste funcional	14
Teste estrutural	14
Teste baseado em defeitos	14
Entrega	14
20. Checklist final	14

1. Sobre a plataforma

O **V&V TestLab** é uma plataforma local para organizar o projeto da disciplina de Verificação e Validação de Software. Ela mantém conectados:

```
Funcionalidade/requisito -> Classe de equivalência -> Caso de teste  
                        -> Execução -> Defeito
```

O projeto aceita no máximo **três funcionalidades ou métodos principais**. Todos os cenários e resultados devem pertencer a esse escopo.

O que a plataforma faz

- Caracteriza o software Python escolhido.
- Organiza as três funcionalidades principais.
- Mantém classes de equivalência válidas e inválidas.
- Gera casos automaticamente por análise do valor limite.
- Registra casos criados manualmente.
- Registra execuções manuais ou importadas do pytest.
- Acompanha cobertura estrutural e mutação.
- Mantém defeitos e propostas de correção.
- Identifica lacunas na matriz de rastreabilidade.
- Exporta CSV, JSON, Markdown e relatório PDF A4.

2. Iniciar a plataforma

Pré-requisitos

- Node.js 20 ou superior.
- npm.
- Um navegador atualizado.
- Python e pytest apenas para usar a integração automática.

Primeira execução

Abra um terminal na pasta do V&V TestLab e execute:

```
npm install  
npm run dev
```

Abra o endereço exibido pelo terminal, normalmente:

```
http://127.0.0.1:5173
```

Use sempre o mesmo endereço. **localhost** e **127.0.0.1** são origens diferentes para o navegador e podem apresentar armazenamentos locais diferentes.

Parar a plataforma

No terminal em que o Vite está sendo executado, pressione:

```
ctrl+c
```

3. Entender a navegação

Tela	Finalidade
Visão geral	Indicadores, evolução, lacunas e etapa atual.
Requisitos	Definição das três funcionalidades/métodos principais.
Cenários e casos	Classes de equivalência, valores-limite e casos detalhados.
Rastreabilidade	Matriz completa entre requisitos, classes, casos, execuções e defeitos.
Execução e métricas	Histórico de resultados, cobertura e mutação.
Integração Python	Execução automática do pytest, cobertura, mutação e sincronização.
Defeitos	Falhas encontradas e propostas de correção.
Dados e exportação	PDF, Markdown, CSV, backup e restauração.

O botão **Configurar**, no canto superior, abre os dados gerais do projeto. O botão **Novo caso** abre rapidamente o cadastro de um caso de teste.

4. Configurar o projeto

Clique em **Configurar** e preencha:

Campo	Como preencher
Nome do software	Nome do projeto Python de terceiros.
Repositório GitHub	URL pública ou privada utilizada no trabalho.
Caminho local do repositório	Pasta em que o código está clonado no computador.
URL da ponte	Normalmente <code>http://127.0.0.1:8765</code> .
Criar defeito ao falhar?	Use "Sim" para abrir defeitos a partir do pytest.
Propósito real	Problema resolvido pelo software e público atendido.
LOC	Total de linhas de código.
Funções e métodos	Quantidade encontrada na caracterização.
Classes	Quantidade de classes no código.
Módulos	Quantidade de módulos Python.
Meta de cobertura	Percentual que o grupo pretende atingir.
Etapa atual	Funcional, Estrutural ou Baseado em defeitos.

Clique em **Salvar**. Os dados são armazenados automaticamente no navegador.

5. Definir as três funcionalidades principais

Abra **Requisitos**. Os dados demonstrativos já ocupam as três posições iniciais. Você pode editar esses itens ou excluí-los e cadastrar os seus.

Editar uma funcionalidade

- 1 Clique no botão de edição do requisito.
- 2 Informe o nome da funcionalidade.
- 3 Informe o método ou função correspondente no código, por exemplo `UserService.create_user`.
- 4 Descreva a regra observável.
- 5 Selecione prioridade e status.
- 6 Clique em **Salvar**.

Substituir uma funcionalidade

- 1 Abra a funcionalidade que será removida.
- 2 Clique em **Excluir**.
- 3 Leia a quantidade de classes, casos, execuções e defeitos afetados.
- 4 Confirme somente se a exclusão estiver correta.
- 5 Clique em **Adicionar funcionalidade** e preencha a nova opção.

A exclusão é feita em cascata para evitar registros órfãos. Faça um backup antes de excluir uma funcionalidade com muitos vínculos.

Limite do escopo

Quando o indicador chegar a **3/3**, a criação de uma quarta funcionalidade será bloqueada. Classes e casos somente poderão usar uma dessas três opções.

6. Criar classes de equivalência

Abra **Cenários e casos** e clique em **Nova classe**.

Preencha:

- **Requisito relacionado:** uma das três funcionalidades.
- **Nome da classe:** nome curto e objetivo.
- **Condição de entrada:** regra que define a partição.
- **Tipo:** válida ou inválida.
- **Limites:** use quando a entrada possuir domínio numérico.

Exemplo:

Campo	Valor
Requisito	REQ-001 - Cadastrar usuário
Nome	Idade permitida
Condição	<code>18 <= idade <= 120</code>

Campo	Valor
Tipo	Válida
Mínimo	18
Máximo	120

Crie também as partições inválidas relevantes, como idade abaixo do mínimo, acima do máximo, campo vazio ou formato incorreto.

7. Gerar casos por valor-limite

Para entradas numéricas, clique em **Gerar por limites**.

- 1 Selecione a funcionalidade.
- 2 Selecione uma classe pertencente à mesma funcionalidade.
- 3 Informe o campo testado.
- 4 Informe mínimo e máximo inclusivos.
- 5 Descreva os resultados esperados para valores válidos e inválidos.
- 6 Clique em **Salvar**.

Para o intervalo [18, 120], a plataforma cria:

17, 18, 19, 119, 120 e 121

Os pontos externos são classificados como inválidos. Valores repetidos são eliminados automaticamente em intervalos pequenos.

8. Criar casos manualmente

Clique em **Novo caso** ou **Adicionar** e preencha:

- Funcionalidade/requisito.
- Classe de equivalência.
- Título do cenário.
- Condição e dados de entrada.
- Pré-condições.
- Passos, um por linha.
- Resultado esperado.
- Validade do cenário.
- Técnica utilizada.
- Prioridade e status.

A plataforma impede que um caso utilize uma classe pertencente a outra funcionalidade.

Técnicas disponíveis

Técnica	Quando utilizar
CE	Particionamento em classes de equivalência.
AVL	Análise do valor limite.
Estrutural	Caso adicionado para cobrir nós, desvios ou caminhos.
Mutação	Caso criado para eliminar um mutante sobrevivente.

9. Pesquisar e filtrar

Os filtros podem ser combinados. A plataforma aplica o operador lógico **E** entre eles.

Exemplo:

```
Validade = Inválido E Técnica = AVL E Requisito = REQ-001
```

Filtros disponíveis

- Requisitos: texto, prioridade e status.
- Classes: texto, tipo e requisito.
- Casos: texto, validade, técnica e requisito.
- Rastreabilidade: texto, requisito, resultado e situação do vínculo.
- Métricas: texto e etapa.
- Execuções: texto, resultado e caso.
- Sincronizações: texto e situação.
- Defeitos: texto, severidade, status e caso.

Clique em **Limpar filtros** para retornar à lista completa.

10. Registrar uma execução manual

Abra **Execução e métricas** e clique em **Nova execução**.

- 1 Selecione o caso executado.
- 2 Informe a data.
- 3 Escolha Aprovado, Falhou ou Bloqueado.
- 4 Identifique o ambiente utilizado.
- 5 Registre o resultado obtido e a evidência textual.
- 6 Clique em **Salvar**.

Se o resultado falhar, abra **Defeitos** e registre o problema encontrado.

11. Registrar cobertura e mutação

Em **Execução e métricas**, clique em **Registrar métrica**.

Etapa funcional

- Registre o tamanho inicial da suíte.
- A cobertura pode ser informada caso já tenha sido medida.
- Mutantes permanecem em zero.

Etapa estrutural

- Execute a suíte com coverage.py ou pytest-cov.
- Registre cobertura de instruções e desvios.
- Adicione casos estruturais até atingir a meta.

Etapa baseada em defeitos

- Execute mutmut ou cosmic-ray pela tela **Integração Python**.
- Confira os mutantes totais, mortos e sobreviventes importados automaticamente.
- Adicione casos com técnica **Mutação**.
- Registre novamente até melhorar o escore.

O escore é calculado por:

```
mutantes mortos / mutantes totais * 100
```

12. Integrar com o repositório Python

A integração permite executar pytest, ler coverage.py e executar Cosmic Ray pela ponte local. O painel cria os registros a partir da resposta de cada execução.

Instalar no ambiente do projeto

```
python -m pip install pytest pytest-cov
# Se Cosmic Ray precisar de dependências incompatíveis, instale-o em outro ambiente
Python.
```

Marcar os testes

```
import pytest

@pytest.mark.vv_case("CT-001")
def test_idade_minima():
    assert cadastrar(idade=18).sucesso
```

Alternativamente, inclua o ID no nome:

```
def test_CT_001_idade_minima():
    ...
```

Iniciar a ponte

No diretório do V&V TestLab:

```
python integration\vv_bridge.py `
  --repo "C:\caminho\repositorio" `
  --port 8765 `
  --cov-source . `
  --mutation-tool cosmic-ray `
  --cosmic-ray-config cosmic-ray.toml `
  --pytest-args tests
```

Se o projeto possui ambiente virtual, execute a ponte com o Python desse ambiente.

Sincronizar

- 1 Mantenha o terminal da ponte aberto.
- 2 Abra **Integração Python**.
- 3 Clique em **Verificar ponte**.
- 4 Confirme o repositório, Python, pytest e cobertura.
- 5 Clique em **Executar e sincronizar**.
- 6 Aguarde o término dos testes.

A plataforma criará:

- Uma execução para cada **CT-xxx** encontrado.
- Uma métrica de cobertura na etapa atual.
- Um registro **SYNC-xxx** no histórico.
- Um defeito para cada caso falho, quando configurado.
- Uma lista de testes sem marcador ou com ID inexistente.

Executar mutação

- 1 Garanta que a suíte pytest está passando.
- 2 Em **Configurar**, escolha **auto**, **mutmut** ou **cosmic-ray**.
- 3 Para Cosmic Ray, informe o arquivo TOML existente dentro do repositório.
- 4 Reinicie a ponte usando o comando atualizado exibido na tela.
- 5 Clique em **Executar mutação**.

A plataforma criará um registro **MUT-xxx** e uma nova linha **MET-xxx** na etapa **Baseado em defeitos**. Rodadas anteriores permanecem no histórico para demonstrar a evolução do escore. No Windows nativo, **auto** prioriza Cosmic Ray; mutmut 3+ deve ser executado no WSL.

Exemplo pronto: sistema de hotel

Na pasta **output/hotel-management-testado**, execute **setup_local.ps1** e **iniciar-integracao.ps1**.

Na raiz do V&V TestLab, execute **npm run dev**. Importe

output/hotel-vvtestlab-projeto-inicial.json: ele contém 3 requisitos (Reservar quartos, Data de entrada, Data de saída), 42 classes e 60 casos, mas nenhuma execução ou métrica. Para ver o estudo completo (sincronizações etapa por etapa no código original e no corrigido, 15 defeitos, 4 métricas, 2 rodadas de mutação, grafos de fluxo de controle e cobertura por caso), importe **output/hotel-vvtestlab-backup.json**.

Demonstração em duas fases: (1) **iniciar-integracao.ps1 -Versao original** e **Executar e sincronizar**: a ponte roda a suíte no código original com **--runxfail**, os casos que revelam defeitos falham e viram registros **DEF-xxx**; (2) encerre a ponte, execute **iniciar-integracao.ps1** (código corrigido) e sincronize de novo: todos os casos passam, com 100% de comandos e desvios nas funções do recorte. Depois clique em **Executar mutação**

(cerca de 5 minutos): o seletor cria os 189 mutantes do recorte e o painel registra o escore. A ponte usa Python 3.9 para pytest e coverage.py e Python 3.12 para Cosmic Ray. Consulte [o roteiro do hotel](#).

Nesta versão, o teste deve ser iniciado pelo botão **Executar e sincronizar**. Uma execução iniciada separadamente pela IDE não é capturada automaticamente.

O manual técnico detalhado está em [INTEGRACAO_PYTEST.md](#).

13. Utilizar a matriz de rastreabilidade

Abra **Rastreabilidade**. Cada linha representa:

```
REQ-xxx -> CE-xxx -> CT-xxx -> EXE-xxx -> DEF-xxx
```

Interpretar lacunas

- **Vínculo ausente na classe:** funcionalidade sem classe de equivalência.
- **Vínculo ausente no caso:** classe ainda não testada.
- **Não executado:** caso cadastrado, mas sem resultado.
- **Sem defeito:** normal para testes aprovados; verifique especialmente falhas sem defeito.

Navegar pela matriz

Clique no requisito, classe, caso, execução ou defeito para abrir o registro correspondente. Isso permite corrigir dados sem procurar novamente em outra tela.

Filtros da matriz

- **Cadeia executada:** mostra linhas com caso e execução.
- **Com lacunas:** mostra itens incompletos.
- **Com defeito:** mostra casos que revelaram defeitos.

14. Registrar defeitos

Abra **Defeitos** e clique em **Novo defeito**.

Preencha:

- Caso que revelou o defeito.
- Título objetivo.
- Descrição e evidência.
- Severidade.
- Status.
- Proposta de correção.

Ao editar, altere o status para **Em correção** ou **Fechado** conforme o andamento. Defeitos importados do pytest aparecem identificados como automáticos.

15. Gerar o relatório final

Abra **Dados e exportação** e clique em **Gerar relatório PDF**.

O relatório A4 retrato contém:

- 1 Capa.
- 2 Resumo executivo.
- 3 Caracterização do software.
- 4 Três funcionalidades principais.
- 5 Metodologia.
- 6 Classes de equivalência.
- 7 Fichas completas dos casos.
- 8 Execuções e suas origens.
- 9 Histórico de sincronizações.
- 10 Evolução da cobertura.
- 11 Defeitos.
- 12 Matriz de rastreabilidade.
- 13 Conclusões e pendências.

Antes de gerar o documento:

- Confira se o projeto possui exatamente três funcionalidades.
- Resolva ou justifique as lacunas da matriz.
- Atualize a etapa atual.
- Registre a última cobertura e mutação.
- Feche ou justifique os defeitos.

Também estão disponíveis:

- **Versão Markdown:** base editável do relatório.
- **Casos em CSV:** tabela compatível com Excel.
- **Backup JSON:** cópia completa e restaurável.

16. Backup e restauração

Criar backup

- 1 Abra **Dados e exportação**.
- 2 Clique em **Backup JSON**.
- 3 Guarde o arquivo `vv-testlab-backup.json` junto aos documentos do trabalho.

Restaurar backup

- 1 Abra **Dados e exportação**.
- 2 Clique em **Selecionar backup**.
- 3 Escolha um JSON gerado pelo V&V TestLab.
- 4 Confirme a substituição dos dados atuais.

Restaurar demonstração

O botão **Restaurar dados** substitui todo o projeto pelo exemplo original. Essa ação deve ser usada apenas após criar um backup.

17. Persistência e privacidade

- Os dados ficam no `localStorage` do navegador.
- O código do repositório não é enviado para serviços externos.
- A ponte aceita conexões somente no computador local.
- Limpar os dados do navegador pode apagar o projeto.
- Use o backup JSON regularmente.

18. Solução de problemas

Os botões não respondem

Atualize com `Ctrl+F5`. Confirme que está executando a versão mais recente com `npm run dev`.

Meus dados desapareceram

Confira se está usando exatamente a mesma URL. `localhost:5173` e `127.0.0.1:5173` possuem armazenamentos diferentes.

Não consigo criar uma funcionalidade

O limite é três. Edite uma existente ou exclua uma funcionalidade antes de cadastrar outra.

A classe não aparece no caso

Primeiro selecione a funcionalidade. A lista mostra somente as classes pertencentes a ela.

A ponte não conecta

- Confirme que o terminal da ponte está aberto.
- Confira a URL configurada.
- Use o mesmo número de porta no comando e na aplicação.
- Verifique se `pytest` está instalado no Python utilizado.

A cobertura não foi importada

Instale `pytest-cov` no mesmo ambiente Python usado para iniciar a ponte e reinicie o serviço.

A mutação não inicia

- Confira em **Verificar ponte** se alguma ferramenta aparece disponível.
- No Windows nativo, use Cosmic Ray ou execute a ponte no WSL para usar `mutmut 3+`.
- Para Cosmic Ray, confirme que o TOML configurado existe dentro do repositório autorizado.

Um teste aparece como não vinculado

Confira se o marcador usa um ID existente:

```
@pytest.mark.vv_case("CT-001")
```

O PDF possui informações incompletas

O relatório reflete exatamente os registros atuais. Volte às telas indicadas, complete os campos e gere novamente.

19. Fluxo recomendado para o trabalho

Fase 1

- Configurar o projeto.
- Registrar métricas do código.
- Definir as três funcionalidades.
- Apresentar contexto, propósito e complexidade.

Teste funcional

- Criar classes válidas e inválidas.
- Gerar valores-limite.
- Completar cenários categóricos manualmente.
- Implementar testes pytest.
- Registrar ou sincronizar as execuções.

Teste estrutural

- Alterar a etapa atual para **Estrutural**.
- Executar e sincronizar a cobertura.
- Inspeccionar caminhos não cobertos.
- Criar casos estruturais.
- Repetir até atingir a meta.

Teste baseado em defeitos

- Alterar a etapa para **Baseado em defeitos**.
- Executar mutmut ou cosmic-ray.
- Registrar os resultados de mutação.
- Criar casos para mutantes sobreviventes.
- Documentar defeitos reais.

Entrega

- Verificar a matriz.
- Resolver ou justificar lacunas.
- Gerar backup JSON.
- Gerar relatório PDF.
- Utilizar os indicadores na apresentação de dois slides.

20. Checklist final

- Pendente - Software Python real de terceiros configurado.

- Pendente - Repositório GitHub informado.
- Pendente - LOC, métodos, classes e módulos registrados.
- Pendente - Exatamente três funcionalidades principais.
- Pendente - Classes válidas e inválidas criadas.
- Pendente - Valores-limite cobertos.
- Pendente - Casos implementados no pytest.
- Pendente - Testes vinculados com **CT-xxx**.
- Pendente - Execuções sincronizadas.
- Pendente - Meta de cobertura atingida ou justificada.
- Pendente - Mutação executada e registrada.
- Pendente - Defeitos documentados e vinculados.
- Pendente - Matriz sem lacunas não justificadas.
- Pendente - Backup JSON salvo.
- Pendente - Relatório PDF revisado.